

```

import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
import numpy as np

# Load dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# Normalize
X_train = X_train / 255.0
X_test = X_test / 255.0

# Flatten
X_train = X_train.reshape(-1, 784)
X_test = X_test.reshape(-1, 784)

# One-hot encoding
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

# Model
model = Sequential([
    Dense(10, activation='softmax', input_shape=(784,))
])

model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])

model.summary()

model.fit(X_train, y_train, epochs=10, batch_size=64)#change the batch size

test_loss, test_acc = model.evaluate(X_test, y_test)
print("Test accuracy:", test_acc)

```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-data/11490434/11490434> — 0s 0us/step
 /usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: Us
 super().__init__(activity_regularizer=activity_regularizer, **kwargs)
 Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 10)	7,850

Total params: 7,850 (30.66 KB)
 Trainable params: 7,850 (30.66 KB)
 Non-trainable params: 0 (0.00 B)

```

Epoch 1/10
938/938 ██████████ 4s 3ms/step - accuracy: 0.7726 - loss: 0.8691
Epoch 2/10
938/938 ██████████ 2s 2ms/step - accuracy: 0.9070 - loss: 0.3386
Epoch 3/10
938/938 ██████████ 2s 2ms/step - accuracy: 0.9152 - loss: 0.3055
Epoch 4/10
938/938 ██████████ 2s 2ms/step - accuracy: 0.9214 - loss: 0.2855
Epoch 5/10
938/938 ██████████ 4s 4ms/step - accuracy: 0.9243 - loss: 0.2721
Epoch 6/10
938/938 ██████████ 4s 2ms/step - accuracy: 0.9270 - loss: 0.2638
Epoch 7/10
938/938 ██████████ 2s 2ms/step - accuracy: 0.9251 - loss: 0.2673
Epoch 8/10
938/938 ██████████ 2s 2ms/step - accuracy: 0.9275 - loss: 0.2606
Epoch 9/10
938/938 ██████████ 2s 3ms/step - accuracy: 0.9303 - loss: 0.2531
Epoch 10/10
938/938 ██████████ 3s 3ms/step - accuracy: 0.9302 - loss: 0.2587
313/313 ██████████ 1s 2ms/step - accuracy: 0.9169 - loss: 0.2998
Test accuracy: 0.9279000163078308

```

```

import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
import numpy as np

# Load dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# Normalize
X_train = X_train / 255.0
X_test = X_test / 255.0

# Flatten
X_train = X_train.reshape(-1, 784)
X_test = X_test.reshape(-1, 784)

# One-hot encoding
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

# Model
model = Sequential([
    Dense(10, activation='softmax', input_shape=(784,))
])

```

```

model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])

model.summary()

model.fit(X_train, y_train, epochs=10, batch_size=32)#change the batch size

test_loss, test_acc = model.evaluate(X_test, y_test)
print("Test accuracy:", test_acc)

```

Model: "sequential_2"

Layer (type)	Output Shape	Param #
dense_2 (Dense)	(None, 10)	7,850

Total params: 7,850 (30.66 KB)

Trainable params: 7,850 (30.66 KB)

Non-trainable params: 0 (0.00 B)

Epoch 1/10

1875/1875 ————— 6s 3ms/step - accuracy: 0.8142 - loss: 0.7173

Epoch 2/10

1875/1875 ————— 4s 2ms/step - accuracy: 0.9132 - loss: 0.3125

Epoch 3/10

1875/1875 ————— 4s 2ms/step - accuracy: 0.9198 - loss: 0.2822

Epoch 4/10

1875/1875 ————— 5s 3ms/step - accuracy: 0.9254 - loss: 0.2695

Epoch 5/10

1875/1875 ————— 4s 2ms/step - accuracy: 0.9262 - loss: 0.2619

Epoch 6/10

1875/1875 ————— 4s 2ms/step - accuracy: 0.9280 - loss: 0.2572

Epoch 7/10

1875/1875 ————— 5s 3ms/step - accuracy: 0.9294 - loss: 0.2586

Epoch 8/10

1875/1875 ————— 4s 2ms/step - accuracy: 0.9290 - loss: 0.2546

Epoch 9/10

1875/1875 ————— 4s 2ms/step - accuracy: 0.9304 - loss: 0.2529

Epoch 10/10

1875/1875 ————— 5s 3ms/step - accuracy: 0.9305 - loss: 0.2475

313/313 ————— 1s 2ms/step - accuracy: 0.9145 - loss: 0.2992

Test accuracy: 0.9264000058174133

```

import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
import numpy as np

# Load dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()

```

```

X_train, y_train, X_test, y_test, validation_data = \

# Normalize
X_train = X_train / 255.0
X_test = X_test / 255.0

# Flatten
X_train = X_train.reshape(-1, 784)
X_test = X_test.reshape(-1, 784)

# One-hot encoding
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

# Model
model = Sequential([
    Dense(10, activation='softmax', input_shape=(784,))
])

model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])

model.summary()

model.fit(X_train, y_train, epochs=10, batch_size=128)#change the batch size

test_loss, test_acc = model.evaluate(X_test, y_test)
print("Test accuracy:", test_acc)

```

Model: "sequential_3"

Layer (type)	Output Shape	Param #
dense_3 (Dense)	(None, 10)	7,850

Total params: 7,850 (30.66 KB)

Trainable params: 7,850 (30.66 KB)

Non-trainable params: 0 (0.00 B)

Epoch 1/10

469/469 ————— 2s 3ms/step - accuracy: 0.7249 - loss: 1.0678

Epoch 2/10

469/469 ————— 1s 3ms/step - accuracy: 0.9008 - loss: 0.3774

Epoch 3/10

469/469 ————— 1s 3ms/step - accuracy: 0.9128 - loss: 0.3226

Epoch 4/10

469/469 ————— 2s 4ms/step - accuracy: 0.9154 - loss: 0.3040

Epoch 5/10

469/469 ————— 2s 5ms/step - accuracy: 0.9197 - loss: 0.2905

Epoch 6/10

469/469 ————— 1s 3ms/step - accuracy: 0.9226 - loss: 0.2822

Epoch 7/10

469/469 ————— 1s 3ms/step - accuracy: 0.9245 - loss: 0.2685

Epoch 8/10

```

Epoch 9/10
469/469 ————— 1s 3ms/step - accuracy: 0.9239 - loss: 0.2693
Epoch 10/10
469/469 ————— 1s 3ms/step - accuracy: 0.9254 - loss: 0.2680
313/313 ————— 1s 2ms/step - accuracy: 0.9129 - loss: 0.3051
Test accuracy: 0.9251999855041504

```

```

import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
import numpy as np

# Load dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# Normalize
X_train = X_train / 255.0
X_test = X_test / 255.0

# Flatten
X_train = X_train.reshape(-1, 784)
X_test = X_test.reshape(-1, 784)

# One-hot encoding
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

# Model
model = Sequential([
    Dense(10, activation='softmax', input_shape=(784,))
])

model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])

model.summary()

model.fit(X_train, y_train, epochs=10, batch_size=256)#change the batch size

test_loss, test_acc = model.evaluate(X_test, y_test)
print("Test accuracy:", test_acc)

```

Model: "sequential_4"

Layer (type)	Output Shape	Param #

dense_4 (Dense)	(None, 10)	7,850
-----------------	------------	-------

Total params: 7,850 (30.66 KB)

Trainable params: 7,850 (30.66 KB)

Non-trainable params: 0 (0.00 B)

Epoch 1/10

235/235 ————— 2s 5ms/step - accuracy: 0.6355 - loss: 1.3252

Epoch 2/10

235/235 ————— 1s 6ms/step - accuracy: 0.8880 - loss: 0.4550

Epoch 3/10

235/235 ————— 1s 5ms/step - accuracy: 0.9007 - loss: 0.3740

Epoch 4/10

235/235 ————— 1s 4ms/step - accuracy: 0.9104 - loss: 0.3319

Epoch 5/10

235/235 ————— 1s 4ms/step - accuracy: 0.9136 - loss: 0.3173

Epoch 6/10

235/235 ————— 1s 4ms/step - accuracy: 0.9175 - loss: 0.3007

Epoch 7/10

235/235 ————— 1s 4ms/step - accuracy: 0.9197 - loss: 0.2923

Epoch 8/10

235/235 ————— 1s 4ms/step - accuracy: 0.9216 - loss: 0.2837

Epoch 9/10

235/235 ————— 1s 4ms/step - accuracy: 0.9237 - loss: 0.2771

Epoch 10/10

235/235 ————— 1s 4ms/step - accuracy: 0.9246 - loss: 0.2742

313/313 ————— 1s 2ms/step - accuracy: 0.9128 - loss: 0.3094

Test accuracy: 0.9241999983787537

```
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
import numpy as np

# Load dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# Normalize
X_train = X_train / 255.0
X_test = X_test / 255.0

# Flatten
X_train = X_train.reshape(-1, 784)
X_test = X_test.reshape(-1, 784)

# One-hot encoding
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

# Model
model = Sequential()
```

```

model = Sequential([
    Dense(10, activation='softmax', input_shape=(784,))
])

model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])

model.summary()

model.fit(X_train, y_train, epochs=15, batch_size=64)#change the number of t

test_loss, test_acc = model.evaluate(X_test, y_test)
print("Test accuracy:", test_acc)

```

Model: "sequential_5"

Layer (type)	Output Shape	Param #
dense_5 (Dense)	(None, 10)	7,850

Total params: 7,850 (30.66 KB)

Trainable params: 7,850 (30.66 KB)

Non-trainable params: 0 (0.00 B)

Epoch 1/15

938/938 ————— 7s 6ms/step - accuracy: 0.7663 - loss: 0.8866

Epoch 2/15

938/938 ————— 4s 4ms/step - accuracy: 0.9084 - loss: 0.3345

Epoch 3/15

938/938 ————— 3s 3ms/step - accuracy: 0.9146 - loss: 0.3055

Epoch 4/15

938/938 ————— 2s 2ms/step - accuracy: 0.9195 - loss: 0.2894

Epoch 5/15

938/938 ————— 4s 4ms/step - accuracy: 0.9224 - loss: 0.2732

Epoch 6/15

938/938 ————— 2s 2ms/step - accuracy: 0.9251 - loss: 0.2715

Epoch 7/15

938/938 ————— 2s 2ms/step - accuracy: 0.9258 - loss: 0.2678

Epoch 8/15

938/938 ————— 2s 3ms/step - accuracy: 0.9283 - loss: 0.2551

Epoch 9/15

938/938 ————— 2s 2ms/step - accuracy: 0.9285 - loss: 0.2577

Epoch 10/15

938/938 ————— 3s 3ms/step - accuracy: 0.9304 - loss: 0.2496

Epoch 11/15

938/938 ————— 2s 3ms/step - accuracy: 0.9302 - loss: 0.2561

Epoch 12/15

938/938 ————— 2s 2ms/step - accuracy: 0.9298 - loss: 0.2516

Epoch 13/15

938/938 ————— 3s 2ms/step - accuracy: 0.9308 - loss: 0.2562

Epoch 14/15

938/938 ————— 2s 2ms/step - accuracy: 0.9316 - loss: 0.2458

Epoch 15/15

938/938 ————— 3s 3ms/step - accuracy: 0.9320 - loss: 0.2432

313/313 ————— 1s 3ms/step - accuracy: 0.9154 - loss: 0.2961

Test accuracy: 0.9266999959945679

```
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
import numpy as np

# Load dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# Normalize
X_train = X_train / 255.0
X_test = X_test / 255.0

# Flatten
X_train = X_train.reshape(-1, 784)
X_test = X_test.reshape(-1, 784)

# One-hot encoding
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

# Model
model = Sequential([
    Dense(10, activation='softmax', input_shape=(784,))
])

model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])

model.summary()

model.fit(X_train, y_train, epochs=10, batch_size=20)#change the batch size

test_loss, test_acc = model.evaluate(X_test, y_test)
print("Test accuracy:", test_acc)
```

Model: "sequential_6"

Layer (type)	Output Shape	Param #
dense_6 (Dense)	(None, 10)	7,850

Total params: 7,850 (30.66 KB)

Trainable params: 7,850 (30.66 KB)

Non-trainable params: 0 (0.00 B)

Epoch 1/10

```

3000/3000 ————— 7s 2ms/step - accuracy: 0.8311 - loss: 0.6463
Epoch 2/10
3000/3000 ————— 8s 3ms/step - accuracy: 0.9176 - loss: 0.3004
Epoch 3/10
3000/3000 ————— 6s 2ms/step - accuracy: 0.9220 - loss: 0.2732
Epoch 4/10
3000/3000 ————— 8s 3ms/step - accuracy: 0.9245 - loss: 0.2666
Epoch 5/10
3000/3000 ————— 7s 2ms/step - accuracy: 0.9273 - loss: 0.2578
Epoch 6/10
3000/3000 ————— 7s 2ms/step - accuracy: 0.9287 - loss: 0.2616
Epoch 7/10
3000/3000 ————— 7s 2ms/step - accuracy: 0.9293 - loss: 0.2554
Epoch 8/10
3000/3000 ————— 7s 2ms/step - accuracy: 0.9307 - loss: 0.2482
Epoch 9/10
3000/3000 ————— 7s 2ms/step - accuracy: 0.9331 - loss: 0.2440
Epoch 10/10
1890/3000 ————— 2s 2ms/step - accuracy: 0.9307 - loss: 0.2468

```

```

# add the accuracy and loss curve
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
import numpy as np
import matplotlib.pyplot as plt

# Load dataset
(X_train, y_train), (X_test, y_test) = tf.keras.datasets.mnist.load_data()

# Normalize
X_train = X_train / 255.0
X_test = X_test / 255.0

# Flatten
X_train = X_train.reshape(-1, 784)
X_test = X_test.reshape(-1, 784)

# One-hot encoding
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

# Model
model = Sequential([
    Dense(10, activation='softmax', input_shape=(784,))
])

model.compile(optimizer='adam',
              loss='categorical_crossentropy',

```

```

        metrics=['accuracy'])

model.summary()

# Train model and store history
history = model.fit(X_train, y_train, epochs=10, batch_size=20, validation_d

# Evaluate model
test_loss, test_acc = model.evaluate(X_test, y_test)
print("Test accuracy:", test_acc)

# Plot Accuracy Curve
plt.figure()
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Model Accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.legend(['Train', 'Test'])
plt.show()

# Plot Loss Curve
plt.figure()
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('Model Loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.legend(['Train', 'Test'])
plt.show()

```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: Us
 super().__init__(activity_regularizer=activity_regularizer, **kwargs)

Model: "sequential_1"

Layer (type)	Output Shape	Param #
dense_1 (Dense)	(None, 10)	7,850

Total params: 7,850 (30.66 KB)

Trainable params: 7,850 (30.66 KB)

Non-trainable params: 0 (0.00 B)

Epoch 1/10

3000/3000 ————— 9s 3ms/step - accuracy: 0.8301 - loss: 0.6466

Epoch 2/10

3000/3000 ————— 9s 2ms/step - accuracy: 0.9167 - loss: 0.3000

Epoch 3/10

3000/3000 ————— 8s 3ms/step - accuracy: 0.9225 - loss: 0.2801

Epoch 4/10

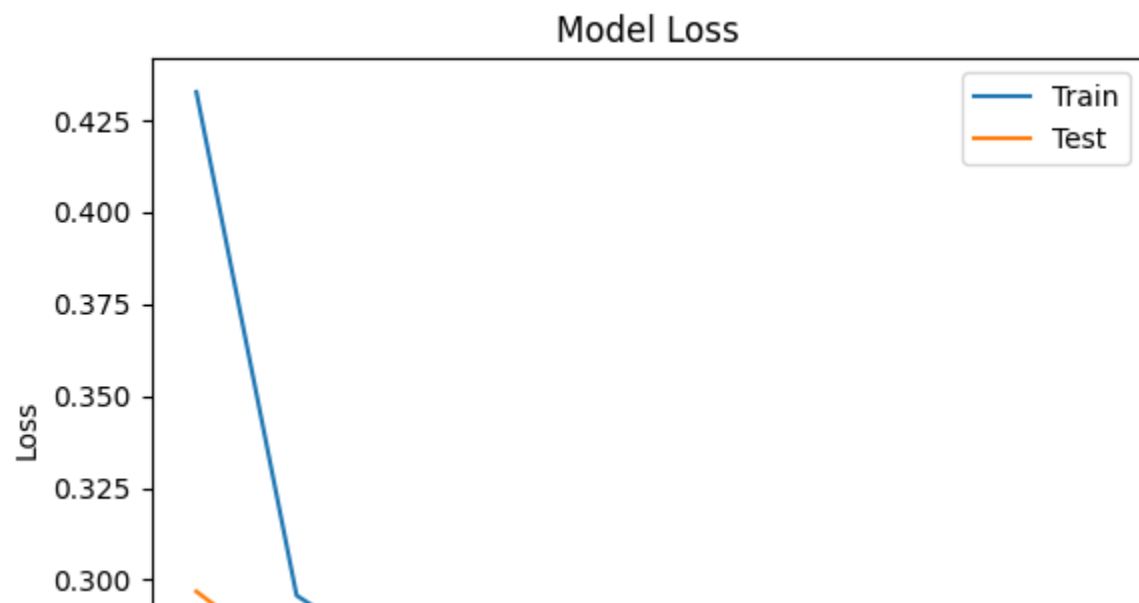
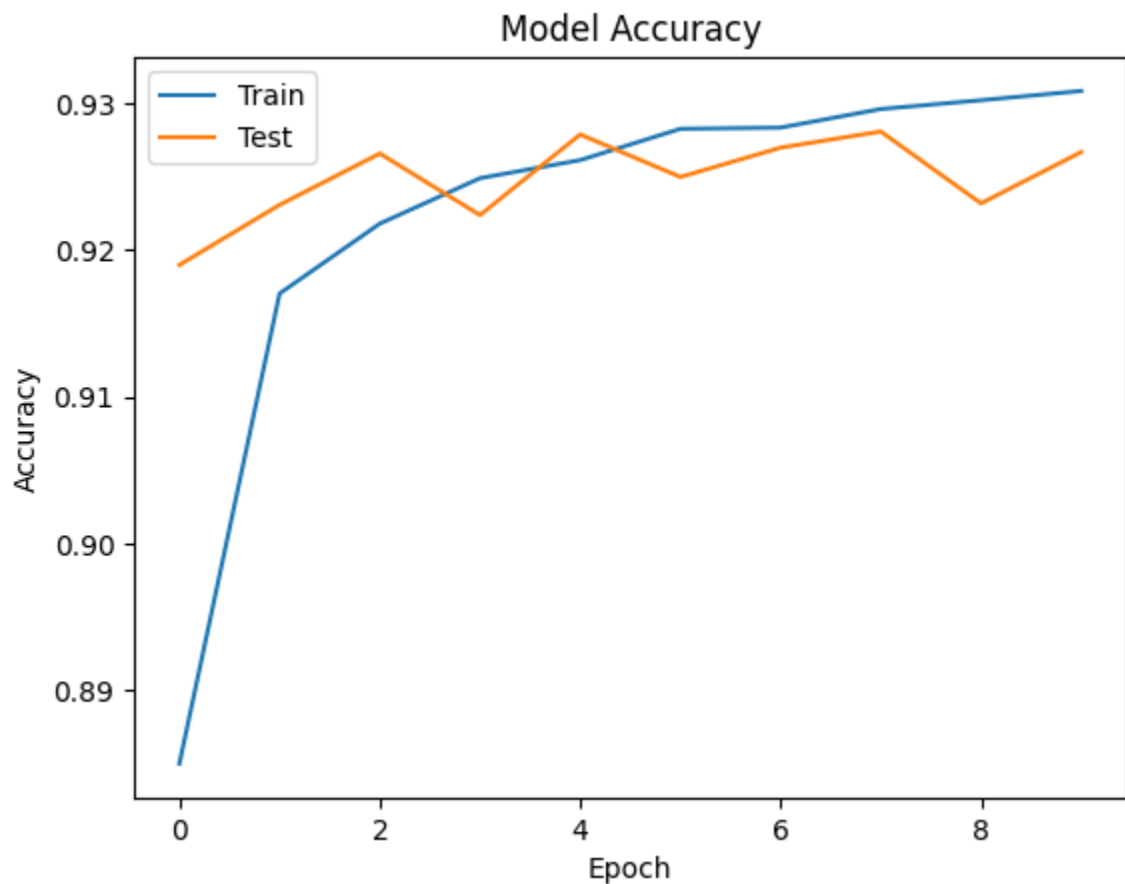
3000/3000 ————— 9s 3ms/step - accuracy: 0.9246 - loss: 0.2672

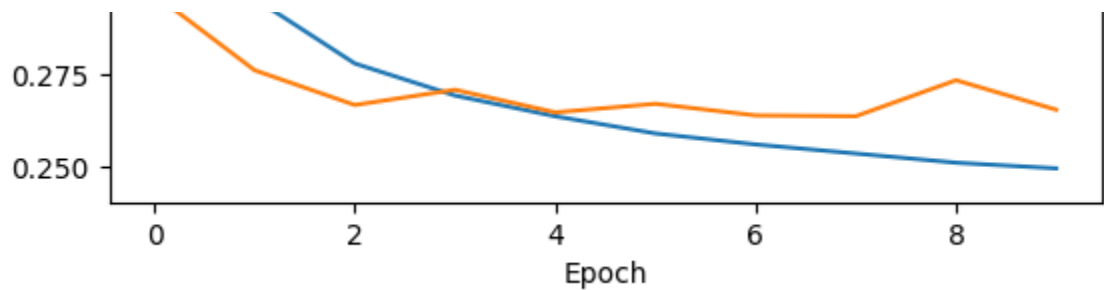
Epoch 5/10

3000/3000 ————— 7s 2ms/step - accuracy: 0.9273 - loss: 0.2611

Epoch 6/10

```
3000/3000 ————— 9s 3ms/step - accuracy: 0.9291 - loss: 0.2561
Epoch 7/10
3000/3000 ————— 7s 2ms/step - accuracy: 0.9283 - loss: 0.2595
Epoch 8/10
3000/3000 ————— 8s 3ms/step - accuracy: 0.9294 - loss: 0.2506
Epoch 9/10
3000/3000 ————— 8s 3ms/step - accuracy: 0.9312 - loss: 0.2501
Epoch 10/10
3000/3000 ————— 7s 2ms/step - accuracy: 0.9298 - loss: 0.2501
313/313 ————— 1s 2ms/step - accuracy: 0.9170 - loss: 0.3007
Test accuracy: 0.9266999959945679
```





```
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
import numpy as np
import matplotlib.pyplot as plt

# Load dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# Normalize
X_train = X_train / 255.0
X_test = X_test / 255.0

# Flatten
X_train = X_train.reshape(-1, 784)
X_test = X_test.reshape(-1, 784)

# One-hot encoding
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

# Model with more layers
model = Sequential([
    Dense(128, activation='relu', input_shape=(784,)),
    Dense(64, activation='relu'),
    Dense(32, activation='relu'),
    Dense(10, activation='softmax')
])

model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])

model.summary()

# Train model
history = model.fit(X_train, y_train,
```

```

        epochs=10,
        batch_size=20,
        validation_data=(X_test, y_test))

# Evaluate model
test_loss, test_acc = model.evaluate(X_test, y_test)
print("Test accuracy:", test_acc)

# # Accuracy Curve
# plt.figure()
# plt.plot(history.history['accuracy'])
# plt.plot(history.history['val_accuracy'])
# plt.title('Model Accuracy')
# plt.ylabel('Accuracy')
# plt.xlabel('Epoch')
# plt.legend(['Train', 'Test'])
# plt.show()

# # Loss Curve
# plt.figure()
# plt.plot(history.history['loss'])
# plt.plot(history.history['val_loss'])
# plt.title('Model Loss')
# plt.ylabel('Loss')
# plt.xlabel('Epoch')
# plt.legend(['Train', 'Test'])
# plt.show()

```

Model: "sequential_8"

Layer (type)	Output Shape	Param #
dense_11 (Dense)	(None, 128)	100,480
dense_12 (Dense)	(None, 64)	8,256
dense_13 (Dense)	(None, 32)	2,080
dense_14 (Dense)	(None, 10)	330

Total params: 111,146 (434.16 KB)

Trainable params: 111,146 (434.16 KB)

Non-trainable params: 0 (0.00 B)

Epoch 1/10

3000/3000 ————— 19s 6ms/step - accuracy: 0.8772 - loss: 0.4072

Epoch 2/10

3000/3000 ————— 17s 6ms/step - accuracy: 0.9655 - loss: 0.1114

Epoch 3/10

3000/3000 ————— 16s 5ms/step - accuracy: 0.9773 - loss: 0.0724

Epoch 4/10

3000/3000 ————— 16s 5ms/step - accuracy: 0.9825 - loss: 0.0555

Epoch 5/10

3000/3000 ————— 16s 5ms/step - accuracy: 0.9863 - loss: 0.0441

Epoch 6/10

```

3000/3000 ————— 17s 6ms/step - accuracy: 0.9890 - loss: 0.0359
Epoch 7/10
3000/3000 ————— 16s 5ms/step - accuracy: 0.9890 - loss: 0.0333
Epoch 8/10
3000/3000 ————— 16s 5ms/step - accuracy: 0.9915 - loss: 0.0269
Epoch 9/10
3000/3000 ————— 16s 5ms/step - accuracy: 0.9916 - loss: 0.0247
Epoch 10/10
3000/3000 ————— 17s 6ms/step - accuracy: 0.9935 - loss: 0.0198
313/313 ————— 1s 3ms/step - accuracy: 0.9618 - loss: 0.1582
Test accuracy: 0.968999981880188

```

```

import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
import matplotlib.pyplot as plt
import numpy as np
from PIL import Image
from google.colab import files

# 1. Load MNIST dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# 2. Normalize pixel values
X_train = X_train / 255.0
X_test = X_test / 255.0

# 3. Flatten images (28x28 → 784)
X_train = X_train.reshape(-1, 784)
X_test = X_test.reshape(-1, 784)

# 4. One-hot encoding
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

# 5. Build model
model = Sequential([
    Dense(128, activation='relu', input_shape=(784,)),
    Dense(64, activation='relu'),
    Dense(32, activation='relu'),
    Dense(10, activation='softmax')
])

# 6. Compile model
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

```

```
        metrics=[ 'accuracy' ]
    )

# 7. Train model
history = model.fit(
    X_train,
    y_train,
    epochs=10,
    batch_size=64,
    validation_data=(X_test, y_test)
)

# 8. Evaluate model
test_loss, test_acc = model.evaluate(X_test, y_test)
print("Test accuracy:", test_acc)

# 9. Upload handwritten image
print("Upload your handwritten digit image")
uploaded = files.upload()

# 10. Load image
filename = list(uploaded.keys())[0]
img = Image.open(filename).convert("L")

# 11. Resize to 28x28
img = img.resize((28,28))

# 12. Convert to numpy
img_array = np.array(img)

# 13. Invert colors (important for MNIST)
img_array = 255 - img_array

# 14. Normalize
img_array = img_array / 255.0

# 15. Flatten
img_array = img_array.reshape(1,784)

# 16. Predict digit
prediction = model.predict(img_array)
digit = np.argmax(prediction)

print("Predicted Digit:", digit)

# 17. Display image
plt.imshow(img_array.reshape(28,28), cmap='gray')
plt.title(f"Predicted Digit: {digit}")
plt.axis('off')
plt.show()
```